

適用於 React 開發人員的 Lit

來源：<https://codelabs.developers.google.com/codelabs/lit-2-for-react-devs?hl=zh-tw>

步驟數：12

在這個程式碼研究室中，您將瞭解如何將 React 概念轉譯為 Lit

目錄

1. 簡介
2. Lit 與 React
3. 設定及探索 Playground
4. JSX 和範本
5. 元件和屬性
6. 狀態和生命週期
7. 吊人胃口的情節片段
8. 兒童
9. Refs
10. 中介狀態
11. 樣式
12. 進階主題 (選用)

1. 簡介

什麼是 Lit

[Lit](#) 是一個簡單的程式庫，可建構快速輕巧的網頁元件，適用於任何框架，或完全不使用框架。您可以使用 Lit 建構可共用的元件、應用程式、設計系統等。

課程內容

如何將多個 React 概念轉換為 Lit，例如：

- JSX 和範本
- 元件和屬性
- 狀態和生命週期
- 吊人胃口的情節片段
- 兒童
- Refs
- 中介狀態

建構項目

完成本程式碼研究室後，您就能將 React 元件概念轉換為對應的 Lit 概念。

軟硬體需求

- 最新版 Chrome、Safari、Firefox 或 Edge。
- 瞭解 HTML、CSS、JavaScript 和 [Chrome 開發人員工具](#)。
- React 知識
- (進階) 如要獲得最佳開發體驗，請下載 [VS Code](#)。您也需要 VS Code 適用的 [lit-plugin](#) 和 [NPM](#)。

不想逐一按照步驟操作嗎？你可以隨意跳轉，不必依序閱讀各節。

步驟 2 · 約 6 分鐘

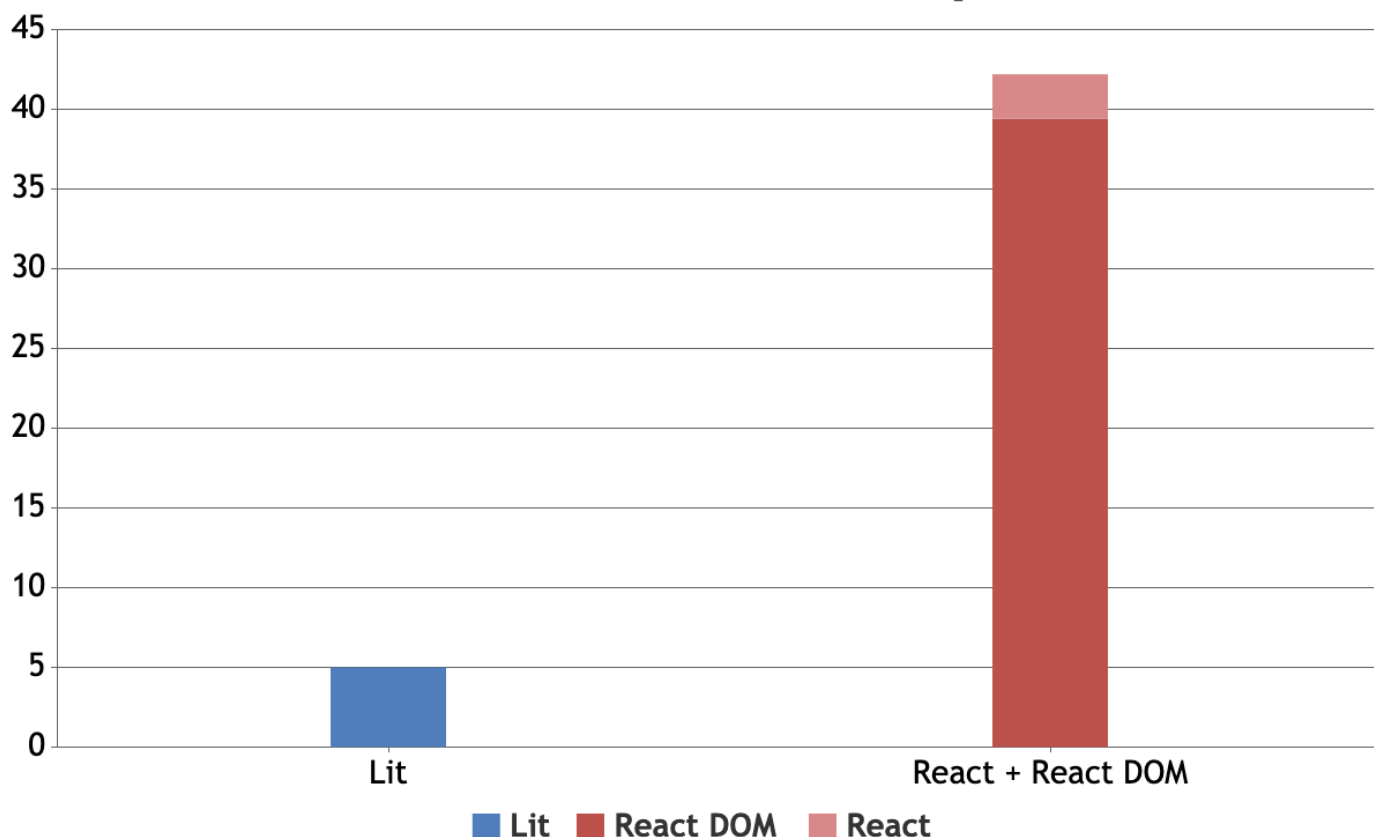
2. Lit 與 React

在許多方面，Lit 的核心概念和功能與 React 相似，但 Lit 也有一些主要差異和區別：

很小

Lit 非常小巧：與 **React + ReactDOM** 的 40 多 KB 相比，縮小並壓縮後約為 5 KB。

Bundle Size Minified + Compressed (kb)

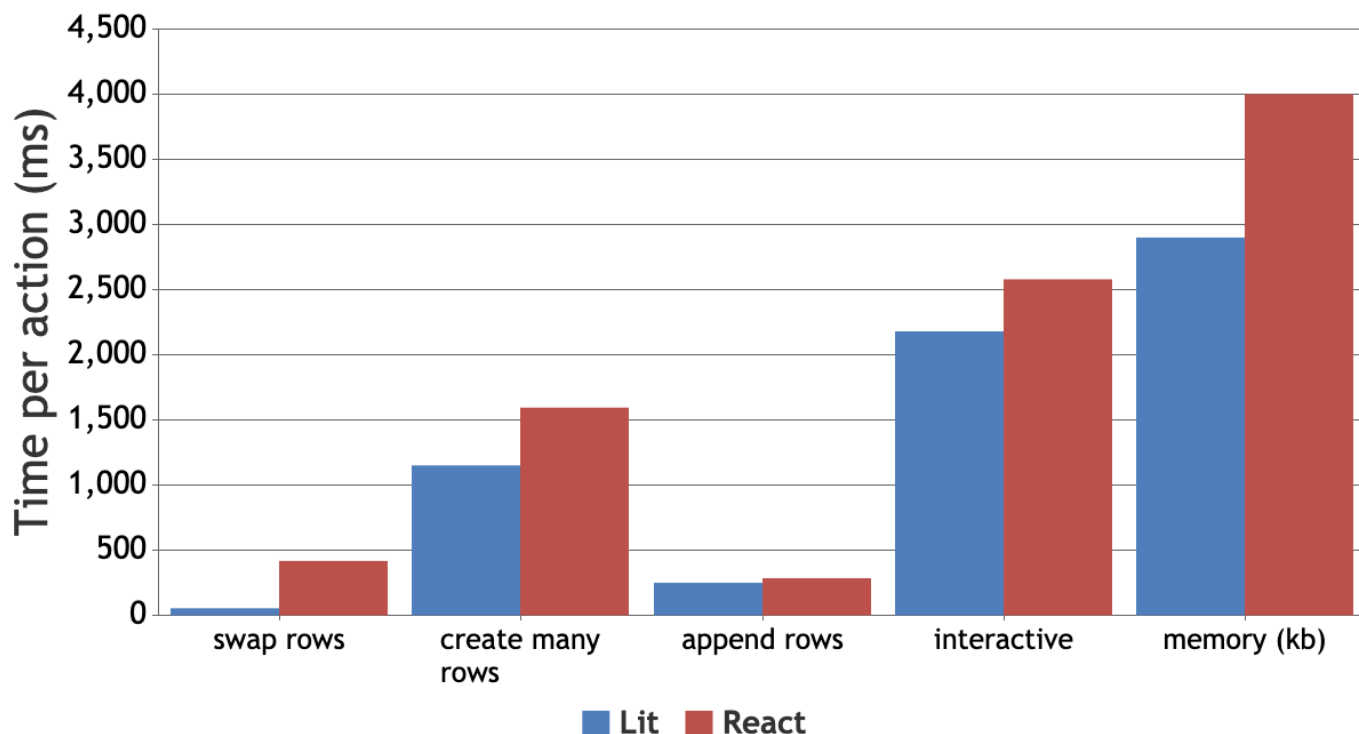


速度很快

在比較 Lit 的範本系統 lit-html 與 React 的 VDOM 的[公開基準](#)中，lit-html 在最差的情況下比 React 快 **8 到 10%**，在更常見的用途下則快 **50%以上**。

LitElement (Lit 的元件基礎類別) 會為 lit-html 增加最少的負擔，但比較記憶體用量、互動和啟動時間等元件功能時，效能比 React 高出 **16% 至 30%**。

List Rendering Performance (lower is better)



不需要建構作業

有了 ES 模組和標記範本字面值等新瀏覽器功能，Lit **不需要編譯即可執行**。也就是說，只要使用指令碼標記、瀏覽器和伺服器，就能設定開發環境並開始運作。

使用 ES 模組和 [Skypack](#) 或 [UNPKG](#) 等現代 CDN，您甚至不需要 NPM 就能開始使用！

不過，您還是可以建構及最佳化 Lit 程式碼。最近開發人員整合原生 ES 模組的趨勢對 Lit 來說是好事，因為 Lit 只是普通的 JavaScript，不需要架構專屬的 CLI 或建構處理程序。

不受框架限制

Lit 的元件是以一組稱為「網頁元件」的網頁標準為基礎建構而成。也就是說，在 Lit 中建構的元件可與目前和未來的架構搭配運作。如果支援 HTML 元素，就支援網頁元件。

架構互通性問題只會在架構對 DOM 的支援有限制時發生。React 就是其中一個框架，但它允許透過參照來規避限制，而 React 中的參照並非良好的開發人員體驗。

Lit 團隊正在進行名為 [@lit-labs/react](#) 的實驗專案，可自動剖析 Lit 元件並產生 React 包裝函式，因此您不必使用參照。

此外，「[Custom Elements Everywhere](#)」也會顯示哪些架構和程式庫可與自訂元素搭配使用！

一流的 TypeScript 支援

雖然您可以使用 JavaScript 編寫所有 Lit 程式碼，但 Lit 是以 TypeScript 編寫，因此 Lit 團隊建議開發人員也使用 TypeScript！

Lit 團隊與 Lit 社群合作，共同維護相關專案，在開發和建構期間，透過 [lit-analyzer](#) 和 [lit-plugin](#)，為 Lit 範本提供 TypeScript 型別檢查和智慧自動完成功能。

```

ame: 'ba
enderLit (attribute) ?outlined: boolean
return Type '5' is not assignable to 'boolean'
<mwC- View Problem (⌘F8) No quick fixes available
?outlined=${5}
value="Hello World">
</mwC-textfield>`;

```

```

return html`
<mwC-textfield
  ?outlined=${true}
  autoValidate (attribute) autoValidate: boolean
  autocalitalize
  charCounter
  disabled
  endAligned
  helper
  helperPersistent
  icon
  iconTrailing
  inputMode
  label
  max

```

瀏覽器內建開發人員工具

Lit 元件只是 DOM 中的 HTML 元素。也就是說，您不必為瀏覽器安裝任何工具或擴充功能，即可檢查元件。

您只要開啟開發人員工具、選取元素，即可探索其屬性或狀態。

 圖片：Chrome 開發人員工具，顯示 \$0 會傳回 <mwC-textfield>、\$0.value 會傳回 hello world、\$0.outlined 會傳回 true，而 {\$0} 則會顯示屬性擴充

專為伺服器端轉譯 (SSR) 而設計

Lit 2 的設計初衷就是支援 SSR。撰寫本程式碼研究室時，Lit 團隊尚未發布穩定版的 SSR 工具，但已在 Google 產品中部署伺服器端算繪元件，並在 React 應用程式中測試 SSR。Lit 團隊預計很快就會在 GitHub 上發布這些工具。

在此期間，您可以前往[這裡](#)追蹤 Lit 團隊的進度。

買入成本低

使用 Lit 不需要投入大量心力！您可以在 Lit 中建構元件，並將其新增至現有專案。**如果不喜歡，您不必一次轉換整個應用程式**，因為網頁元件適用於其他架構！

您是否已在 Lit 中建構整個應用程式，但想改用其他項目？那麼，您可以將目前的 Lit 應用程式放在新架構中，並將任何內容遷移至新架構的元件。

此外，[許多現代架構都支援以網頁元件輸出](#)，因此通常可以自行納入 Lit 元素。

3. 設定及探索 Playground

您可以透過兩種方式完成本程式碼研究室：

- 您可以在瀏覽器中完全線上完成
- (進階) 您可以使用 VS Code 在本機執行這項操作

存取程式碼

本程式碼研究室會提供 Lit Playground 的連結，如下所示：

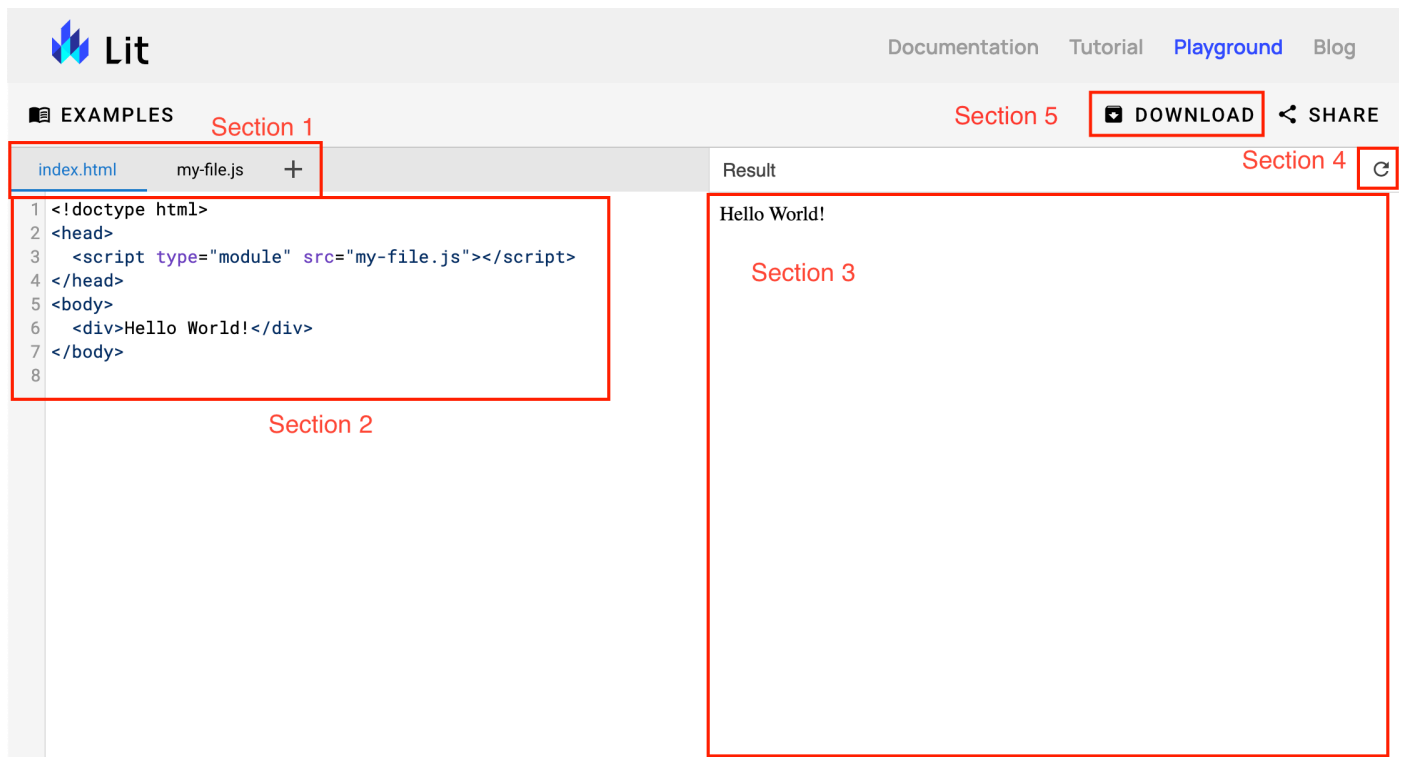
這個遊樂區是程式碼沙箱，完全在瀏覽器中執行。它可以編譯及執行 TypeScript 和 JavaScript 檔案，也能自動將匯入內容解析為節點模組。例如：

```
// before
import './my-file.js';
import 'lit';

// after
import './my-file.js';
import 'https://cdn.skypack.dev/lit';
```

您可以在 Lit Playground 中完成整個教學課程，並將這些檢查點做為起點。如果您使用 VS Code，可以透過這些檢查點下載任何步驟的起始程式碼，並用來檢查您的成果。

探索點亮的遊樂場 UI



Lit 操場 UI 螢幕截圖會標示您在本程式碼研究室中使用的區段。

1. 檔案選取器。請注意加號按鈕...
2. 檔案編輯器。

3. 程式碼預覽。
4. 重新載入按鈕。
5. 「下載」按鈕。

VS Code 設定 (進階)

如果您打算完全在瀏覽器中執行本程式碼研究室，可以略過這個步驟。

使用這項 VS Code 設定的好處如下：

- 檢查範本類型
- 範本智慧感應和自動完成

如果您已安裝 NPM、VS Code (含 [lit-plugin](#) 外掛程式)，且知道如何使用該環境，只要下載並啟動這些專案即可，方法如下：

- 按下下載按鈕
- 將 tar 檔案內容解壓縮至目錄
- (如果是 TS) 設定[快速 tsconfig](#)，輸出 ES 模組和 ES2015 以上版本
- 安裝可解析裸露模組指定符的開發伺服器 (Lit 團隊建議使用 [@web/dev-server](#))
 - 範例如下：`package.json`
- 執行開發伺服器並開啟瀏覽器 (如果您使用 [@web/dev-server](#)，可以按 `npx web-dev-server --node-resolve --watch --open`)
 - 如果您使用範例 `package.json`，請使用 `npm run dev`

步驟 4 · 約 13 分鐘

4. JSX 和範本

本節將說明 Lit 中的範本基本概念。

JSX 和 Lit 範本

JSX 是 JavaScript 的語法擴充功能，可讓 React 使用者輕鬆在 JavaScript 程式碼中編寫範本。[Lit 範本](#)的用途類似，都是將元件的 UI 表示為其狀態的函式。

基本語法

在 React 中，您會像這樣算繪 JSX Hello World：

```
import 'react';
import ReactDOM from 'react-dom';

const name = 'Josh Perez';
const element = (
  <>
    <h1>Hello, {name}</h1>
    <div>How are you?</div>
  </>
);

ReactDOM.render(
  element,
  mountNode
);
```

在上述範例中，有兩個元素和一個納入的「名稱」變數。在 Lit 中，您會執行下列操作：

```
import {html, render} from 'lit';

const name = 'Josh Perez';
const element = html`
  <h1>Hello, ${name}</h1>
  <div>How are you?</div>`;

render(
  element,
  mountNode
);
```

部分編輯器不支援標記範本常值的語法螢光標示。因此，Lit 團隊建議使用 Lit Playground 或 VS Code 搭配 lit-plugin。

請注意，Lit 範本不需要 React Fragment，即可在範本中將多個元素分組。

在 Lit 中，範本會以 `html` 標記範本 **LIT**eral 包裝，這也是 Lit 名稱的由來！

範本值

Lit 範本可以接受其他 Lit 範本，也就是 `TemplateResult`。舉例來說，請將 `name` 包在斜體 (`<i>`) 標記中，並以標記範本常值 *N.B.* 包住。請務必使用倒引號字元 (```)，而非單引號字元 (`'`)。

```
import {html, render} from 'lit';

const name = html`<i>Josh Perez</i>`;
const element = html`
  <h1>Hello, ${name}</h1>
  <div>How are you?</div>`;

render(
  element,
  mountNode
);
```

Lit `TemplateResult` 可接受陣列、字串、其他 `TemplateResult`，以及指令。

請嘗試將下列 React 程式碼轉換為 Lit，做為練習：

```
const itemsToBuy = [
  <li>Bananas</li>,
  <li>oranges</li>,
  <li>apples</li>,
  <li>grapes</li>
];

const element = (
  <>
    <h1>Things to buy:</h1>
    <ol>
      {itemsToBuy}
    </ol>
  </>
);

ReactDOM.render(
  element,
  mountNode
);
```

答案：

```
import {html, render} from 'lit';

const itemsToBuy = [
  html`<li>Bananas</li>`,
  html`<li>oranges</li>`,
  html`<li>apples</li>`,
  html`<li>grapes</li>`
];

const element = html`
  <h1>Things to buy:</h1>
  <ol>
    ${itemsToBuy}
  </ol>`;

render(
  element,
  mountNode
);
```

傳遞及設定屬性

JSX 和 Lit 語法最大的差異之一是資料繫結語法。舉例來說，請看這個含有繫結的 React 輸入內容：

```
const disabled = false;
const label = 'my label';
const myClass = 'my-class';
const value = 'my value';
const element =
  <input
    disabled={disabled}
    className={`static-class ${myClass}`}
    defaultValue={value}/>;

ReactDOM.render(
  element,
  mountNode
);
```

在上述範例中，定義的輸入內容會執行下列操作：

- 將 `disabled` 設為已定義的變數 (在此情況下為 `false`)
- 將類別設為 `static-class` 加上變數 (在本例中為 `"static-class my-class"`)
- 設定預設值

在 Lit 中，您會執行下列操作：

```
import {html, render} from 'lit';

const disabled = false;
const label = 'my label';
const myClass = 'my-class';
const value = 'my value';
const element = html`
  <input
    ?disabled=${disabled}
    class="static-class ${myClass}"
    .value=${value}>`;

render(
  element,
  mountNode
);
```

在 Lit 範例中，系統會新增布林值繫結，切換 `disabled` 屬性。

接著，直接繫結至 `class` 屬性，而非 `className`。除非您使用 `classMap` 指令 (用於切換類別的宣告式輔助程式)，否則可以在 `class` 屬性中新增多個繫結。

最後，輸入內容會設定 `value` 屬性。與 React 不同，這不會將輸入元素設為唯讀，因為這會遵循輸入的原始實作和行為。

Lit 屬性繫結語法

```
html`<my-element ?attribute-name=${booleanVar}>`;
```

- `?` 前置字串是切換元素屬性的繫結語法
- 等同於 `inputRef.toggleAttribute('attribute-name', booleanVar)`
- 適用於使用 `disabled` 的元素，因為 DOM 仍會將 `disabled="false"` 讀取為 `true`，`inputElement.hasAttribute('disabled') === true`

```
html`<my-element .property-name=${anyVar}>`;
```

- `.` 前置字串是設定元素屬性的繫結語法
- 等同於 `inputRef.propertyName = anyVar`
- 適合傳遞物件、陣列或類別等複雜資料

```
html`<my-element attribute-name=${stringVar}>`;
```

- 繫結至元素的屬性
- 等同於 `inputRef.setAttribute('attribute-name', stringVar)`
- 適合用於基本值、樣式規則選取器和 `querySelector`s

傳遞處理常式

```

const disabled = false;
const label = 'my label';
const myClass = 'my-class';
const value = 'my value';
const element =
  <input
    onClick={() => console.log('click')}
    onChange={e => console.log(e.target.value)} />;

ReactDOM.render(
  element,
  mountNode
);

```

在上述範例中，定義的輸入內容會執行下列操作：

- 在點選輸入內容時記錄「click」一字
- 在使用者輸入字元時記錄輸入值

在 Lit 中，您會執行下列操作：

```

import {html, render} from 'lit';

const disabled = false;
const label = 'my label';
const myClass = 'my-class';
const value = 'my value';
const element = html`
  <input
    @click=${() => console.log('click')}
    @input=${e => console.log(e.target.value)}>`;

render(
  element,
  mountNode
);

```

在 Lit 範例中，系統會將監聽器新增至 `click` 事件，並使用 `@click`。

接著，由於原生 `change` 事件只會在 `blur` 上觸發 (React 會抽象化這些事件)，因此請改為繫結至 `<input>` 的原生 `input` 事件，而非使用 `onChange`。

Lit 事件處理常式語法

```
html`<my-element @event-name=${() => {...}}></my-element>`;
```

- `@` 前置字串是事件監聽器的繫結語法
- 等同於 `inputRef.addEventListener('event-name', ...)`
- 使用原生 DOM 事件名稱

步驟 5 · 約 9 分鐘

5. 元件和屬性

本節將說明 Lit 類別元件和函式。我們會在後續章節中詳細說明 State 和 Hooks。

類別元件和 LitElement

React 類別元件的 Lit 對等項目是 LitElement，而 Lit 的「反應式屬性」概念則是 React 屬性和狀態的組合。例如：

```
import React from 'react';
import ReactDOM from 'react-dom';

class Welcome extends React.Component {
  constructor(props) {
    super(props);
    this.state = {name: ''};
  }

  render() {
    return <h1>Hello, {this.props.name}</h1>;
  }
}

const element = <Welcome name="Elliott"/>
ReactDOM.render(
  element,
  mountNode
);
```

在上述範例中，React 元件會執行下列操作：

- 轉譯 `name`
- 將 `name` 的預設值設為空字串 (`""`)
- 將 `name` 重新指派給 `"Elliott"`

在 LitElement 中，您可以這樣做：

在 TypeScript 中：

```
import {LitElement, html} from 'lit';
import {customElement, property} from 'lit/decorators.js';

@customElement('welcome-banner')
class WelcomeBanner extends LitElement {
  @property({type: String})
  name = '';

  render() {
    return html`<h1>Hello, ${this.name}</h1>`;
  }
}
```


在 JavaScript 中：

```
import {LitElement, html} from 'lit';

class WelcomeBanner extends LitElement {
  static get properties() {
    return {
      name: {type: String}
    }
  }

  constructor() {
    super();
    this.name = '';
  }

  render() {
    return html`<h1>Hello, ${this.name}</h1>`;
  }
}

customElements.define('welcome-banner', WelcomeBanner);
```

HTML 檔案：

```
<!-- index.html -->
<head>
  <script type="module" src="./index.js"></script>
</head>
<body>
  <welcome-banner name="Elliott"></welcome-banner>
</body>
```

上述範例的審查結果如下：

```
@property({type: String})
name = '';
```

- 定義公開反應式屬性，也就是元件公用 API 的一部分
- 公開屬性 (預設) 和元件的屬性
- 定義如何將元件的屬性 (字串) 轉換為值

反應式屬性是類別屬性，變更時會觸發算繪生命週期。這些項目是由 `@property` 或 `@state` 定義。如要進一步瞭解反應式屬性，請參閱「[狀態和生命週期](#)」一節。

```
static get properties() {  
  return {  
    name: {type: String}  
  }  
}
```

- 這項裝飾器與 `@property` TS 裝飾器功能相同，但會在 JavaScript 中以原生方式執行

```
render() {  
  return html`<h1>Hello, ${this.name}</h1>`  
}
```

- 每當任何反應式屬性變更時，系統就會呼叫這個函式

```
@customElement('welcome-banner')  
class WelcomeBanner extends LitElement {  
  ...  
}
```

- 這會將 HTML 元素標記名稱與類別定義建立關聯
- 根據自訂元素標準，標記名稱必須包含連字號 (-)
- `this` 中的 `LitElement` 是指自訂元素 (本例中為 `<welcome-banner>`) 的執行個體

```
customElements.define('welcome-banner', WelcomeBanner);
```

- 這是 JavaScript 等效項目，相當於 `@customElement` TS 裝飾器

```
<head>  
  <script type="module" src="./index.js"></script>  
</head>
```

- 匯入自訂元素定義

```
<body>  
  <welcome-banner name="Elliott"></welcome-banner>  
</body>
```

- 將自訂元素新增至網頁
- 將 `name` 屬性設為 `'Elliott'`

您也可以使用 Lit 範本、`document.body.innerHTML` 或其他 DOM API，將這個元素新增至主體

函式元件

Lit 沒有函式元件的 1:1 解譯，因為它不使用 JSX 或前置處理器。不過，您可以輕鬆組合函式，根據屬性轉譯 DOM。例如：

```
function Welcome(props) {
  return <h1>Hello, {props.name}</h1>;
}

const element = <Welcome name="Elliott"/>
ReactDOM.render(
  element,
  mountNode
);
```

在 Lit 中，這會是：

```
import {html, render} from 'lit';

function Welcome(props) {
  return html`<h1>Hello, ${props.name}</h1>`;
}

render(
  Welcome({name: 'Elliott'}),
  document.body.querySelector('#root')
);
```

在 Lit 中，用於算繪的函式只能做為輔助函式。一般來說，將函式建構為獨立元件違反慣例，因為函式沒有生命週期，且缺少自訂元素提供的互通性。

6. 狀態和生命週期

在本節中，您將瞭解 Lit 的狀態和生命週期。

州

Lit 的「反應式屬性」概念混合了 React 的狀態和屬性。變更反應式屬性時，可以觸發元件生命週期。反應式屬性有兩種變體：

公開反應式屬性

```
// React
import React from 'react';

class MyEl extends React.Component {
  constructor(props) {
    super(props)
    this.state = {name: 'there'}
  }

  componentWillReceiveProps(nextProps) {
    if (this.props.name !== nextProps.name) {
      this.setState({name: nextProps.name})
    }
  }
}

// Lit (TS)
import {LitElement} from 'lit';
import {property} from 'lit/decorators.js';

class MyEl extends LitElement {
  @property() name = 'there';
}
```

- 定義者：@property
- 類似於 React 的 props 和 state，但可變動
- 元件消費者存取及設定的公用 API

內部反應狀態

```
// React
import React from 'react';

class MyEl extends React.Component {
  constructor(props) {
    super(props)
    this.state = {name: 'there'}
  }
}

// Lit (TS)
import {LitElement} from 'lit';
import {state} from 'lit/decorators.js';

class MyEl extends LitElement {
  @state() name = 'there';
}
```

- 定義者：`@state`
- 類似於 React 的狀態，但可變動
- 私有內部狀態，通常可從元件或子類別中存取

生命週期

Lit 的生命週期與 React 的生命週期非常相似，但也有一些顯著差異。

constructor

```

// React (js)
import React from 'react';

class MyEl extends React.Component {
  constructor(props) {
    super(props);
    this.state = { counter: 0 };
    this._privateProp = 'private';
  }
}

// Lit (ts)
class MyEl extends LitElement {
  @property({type: Number}) counter = 0;
  private _privateProp = 'private';
}

// Lit (js)
class MyEl extends LitElement {
  static get properties() {
    return { counter: {type: Number} }
  }
  constructor() {
    this.counter = 0;
    this._privateProp = 'private';
  }
}

```

- Lit 等效項目也是 `constructor`
- 不需要將任何內容傳遞至超級呼叫
- 呼叫者 (不完全包含) :
 - `document.createElement`
 - `document.innerHTML`
 - `new ComponentClass()`
 - 如果網頁上有名稱未升級的代碼，且定義已載入並註冊 `@customElement` 或 `customElements.define`
- 功能與 React 的 `constructor` 類似

render

```

// React
render() {
  return <div>Hello World</div>
}

// Lit
render() {
  return html`<div>Hello World</div>`;
}

```

- Lit 等效項目也是 `render`
- 可傳回任何可算繪的結果，例如 `TemplateResult` 或 `string` 等。
- 與 `React` 類似，`render()` 應為純函式
- 會算繪至 `createRenderRoot()` 傳回的任何節點 (預設為 `ShadowRoot`)

`componentDidMount`

`componentDidMount` 類似於 Lit 的 `firstUpdated` 和 `connectedCallback` 生命週期回呼的組合。

`firstUpdated`

```
import Chart from 'chart.js';

// React
componentDidMount() {
  this._chart = new Chart(this.chartElRef.current, {...});
}

// Lit
firstUpdated() {
  this._chart = new Chart(this.chartEl, {...});
}
```

- 元件範本首次算繪至元件根目錄時呼叫
- 只有在元素已連線時才會呼叫，例如在節點附加至 DOM 樹狀結構之前，不會透過 `document.createElement('my-component')` 呼叫
- 適合用來執行需要元件所算繪 DOM 的元件設定
- 與 `React` 不同，`componentDidMount` 中反應式屬性的變更會導致重新轉譯，但瀏覽器通常會將變更批次處理到同一個影格中。`firstUpdated` 如果這些變更不需要存取根的 DOM，通常應放在 `willUpdate`

`connectedCallback`

```
// React
componentDidMount() {
  this.window.addEventListener('resize', this.boundOnResize);
}

// Lit
connectedCallback() {
  super.connectedCallback();
  this.window.addEventListener('resize', this.boundOnResize);
}
```

- 每當自訂元素插入 DOM 樹狀結構時呼叫
- 與 `React` 元件不同，自訂元素從 DOM 分離時不會遭到破壞，因此可以「連結」多次
 - 系統不會再次呼叫 `firstUpdated`
- 有助於重新初始化 DOM，或重新附加在中斷連線時清除的事件監聽器
- 注意：`connectedCallback` 可能會在 `firstUpdated` 之前呼叫，因此在第一次呼叫時，DOM 可能無法使用

componentDidUpdate

```
// React
componentDidUpdate(prevProps) {
  if (this.props.title !== prevProps.title) {
    this._chart.setTitle(this.props.title);
  }
}

// Lit (ts)
updated(prevProps: PropertyValues<this>) {
  if (prevProps.has('title')) {
    this._chart.setTitle(this.title);
  }
}
```

- Lit 對等項目為 `updated` (使用「update」的英文過去式)
- 與 React 不同的是，`updated` 也會在初始算繪時呼叫
- 功能與 React 的 `componentDidUpdate` 類似

不想在反應式屬性變更時觸發重新算繪嗎？如需 `willUpdate` 和 `update`，請參閱「[進階主題 - 狀態和生命週期](#)」。

componentWillUnmount

```
// React
componentWillUnmount() {
  this.window.removeEventListener('resize', this.boundOnResize);
}

// Lit
disconnectedCallback() {
  super.disconnectedCallback();
  this.window.removeEventListener('resize', this.boundOnResize);
}
```

- Lit 對等項目與 `disconnectedCallback` 類似
- 與 React 元件不同，自訂元素從 DOM 卸離時，元件不會遭到刪除
- 與 `componentWillUnmount` 不同，`disconnectedCallback` 是在元素從樹狀結構中移除後呼叫
- 根目錄內的 DOM 仍會附加至根目錄的子樹狀結構
- 有助於清除事件監聽器和記憶體洩漏的參照，讓瀏覽器可以對元件進行垃圾收集

運動


```

import React from 'react';
import ReactDOM from 'react-dom';

class Clock extends React.Component {
  constructor(props) {
    super(props);
    this.state = {date: new Date()};
  }

  componentDidMount() {
    this.timerID = setInterval(
      () => this.tick(),
      1000
    );
  }

  componentWillUnmount() {
    clearInterval(this.timerID);
  }

  tick() {
    this.setState({
      date: new Date()
    });
  }

  render() {
    return (
      <div>
        <h1>Hello, world!</h1>
        <h2>It is {this.state.date.toLocaleTimeString()}.</h2>
      </div>
    );
  }
}

ReactDOM.render(
  <Clock />,
  document.getElementById('root')
);

```

在上述範例中，簡單時鐘會執行下列操作：

- 並顯示「Hello World! 「現在時間是」 並顯示時間
- 每秒更新時鐘
- 卸載時，系統會清除呼叫滴答的間隔

首先，從元件類別宣告開始：

```
// Lit (TS)
// some imports here are imported in advance
import {LitElement, html} from 'lit';
import {customElement, state} from 'lit/decorators.js';

@customElement('lit-clock')
class LitClock extends LitElement {
}

// Lit (JS)
// `html` is imported in advance
import {LitElement, html} from 'lit';

class LitClock extends LitElement {
}

customElements.define('lit-clock', LitClock);
```

接著，初始化 `date`，並使用 `@state` 將其宣告為內部反應式屬性，因為元件使用者不會直接設定 `date`。

雖然某些瀏覽器支援 JavaScript 中的私有屬性，但為了簡化，我們不會使用這些屬性，也不會使用這個範例中的 `_` 前置字元屬性。

```

// Lit (TS)
import {LitElement, html} from 'lit';
import {customElement, state} from 'lit/decorators.js';

@customElement('lit-clock')
class LitClock extends LitElement {
  @state() // declares internal reactive prop
  private date = new Date(); // initialization
}

// Lit (JS)
import {LitElement, html} from 'lit';

class LitClock extends LitElement {
  static get properties() {
    return {
      // declares internal reactive prop
      date: {state: true}
    }
  }

  constructor() {
    super();
    // initialization
    this.date = new Date();
  }
}

customElements.define('lit-clock', LitClock);

```

接著，算繪範本。

建議您依生命週期呼叫順序排列 Lit 方法。在本例中，順序為 `properties`、`constructor`，然後是 `render`。

```

// Lit (JS & TS)
render() {
  return html`
    <div>
      <h1>Hello, World!</h1>
      <h2>It is ${this.date.toLocaleTimeString()}.</h2>
    </div>
  `;
}

```

現在，請實作勾號方法。

```
tick() {
  this.date = new Date();
}
```

接著是 `componentDidMount` 的實作。同樣地，Lit 類似項目是 `firstUpdated` 和 `connectedCallback` 的混合體。就這個元件而言，使用 `setInterval` 呼叫 `tick` 時，不需要存取根層級內的 DOM。此外，從文件樹狀結構移除元素時，間隔也會清除，因此如果重新附加元素，間隔就必須重新開始。因此，`connectedCallback` 是較好的選擇。

```
// Lit (TS)
@customElement('lit-clock')
class LitClock extends LitElement {
  @state()
  private date = new Date();
  // initialize timerId for TS
  private timerId = -1 as unknown as ReturnType<typeof setTimeout>;

  connectedCallback() {
    super.connectedCallback();
    this.timerId = setInterval(
      () => this.tick(),
      1000
    );
  }

  ...
}

// Lit (JS)
constructor() {
  super();
  // initialization
  this.date = new Date();
  this.timerId = -1; // initialize timerId for JS
}

connectedCallback() {
  super.connectedCallback();
  this.timerId = setInterval(
    () => this.tick(),
    1000
  );
}
```

最後，請清理間隔，以免元素與文件樹狀結構中斷連線後，系統仍執行勾號。

```
// Lit (TS & JS)
disconnectedCallback() {
  super.disconnectedCallback();
  clearInterval(this.timerId);
}
```

總而言之，最終結果應如下所示：

```

// Lit (TS)
import {LitElement, html} from 'lit';
import {customElement, state} from 'lit/decorators.js';

@customElement('lit-clock')
class LitClock extends LitElement {
  @state()
  private date = new Date();
  private timerId = -1 as unknown as ReturnType<typeof setTimeout>;

  connectedCallback() {
    super.connectedCallback();
    this.timerId = setInterval(
      () => this.tick(),
      1000
    );
  }

  tick() {
    this.date = new Date();
  }

  render() {
    return html`
      <div>
        <h1>Hello, World!</h1>
        <h2>It is ${this.date.toLocaleTimeString()}</h2>
      </div>
    `;
  }

  disconnectedCallback() {
    super.disconnectedCallback();
    clearInterval(this.timerId);
  }
}

// Lit (JS)
import {LitElement, html} from 'lit';

class LitClock extends LitElement {
  static get properties() {
    return {
      date: {state: true}
    }
  }

  constructor() {
    super();
    this.date = new Date();
  }
}

```

```
connectedCallback() {
  super.connectedCallback();
  this.timerId = setInterval(
    () => this.tick(),
    1000
  );
}

tick() {
  this.date = new Date();
}

render() {
  return html`
    <div>
      <h1>Hello, World!</h1>
      <h2>It is ${this.date.toLocaleTimeString()}</h2>
    </div>
  `;
}

disconnectedCallback() {
  super.disconnectedCallback();
  clearInterval(this.timerId);
}
}

customElements.define('lit-clock', LitClock);
```

7. 吊人胃口的情節片段

在本節中，您將瞭解如何將 React Hook 概念轉換為 Lit。

React 鉤子的概念

React Hook 可讓函式元件「連結」至狀態。這麼做有幾個好處。

- 可簡化有狀態邏輯的重複使用
- 協助將元件分割為較小的函式

此外，著重於函式型元件可解決 React 類別型語法的特定問題，例如：

- 必須將 `props` 從 `constructor` 傳遞至 `super`
- `constructor`
 - 中的屬性初始化作業不整齊
 - 這是 React 團隊當時提出的原因，但 ES2019 已解決這個問題
- `this` 不再參照元件所導致的問題

Lit 中的 React 函式庫概念

如「元件和屬性」一節所述，Lit 無法透過函式建立自訂元素，但 `LitElement` 可解決 React 類別元件的大部分主要問題。例如：


```
// React (at the time of making hooks)
import React from 'react';
import ReactDOM from 'react-dom';

class MyEl extends React.Component {
  constructor(props) {
    super(props); // Leaky implementation
    this.state = {count: 0};
    this._chart = null; // Deemed messy
  }

  render() {
    return (
      <>
        <div>Num times clicked {count}</div>
        <button onClick={this.clickCallback}>click me</button>
      </>
    );
  }

  clickCallback() {
    // Errors because `this` no longer refers to the component
    this.setState({count: this.count + 1});
  }
}

// Lit (ts)
class MyEl extends LitElement {
  @property({type: Number}) count = 0; // No need for constructor to set state
  private _chart = null; // Public class fields introduced to JS in 2019

  render() {
    return html`
      <div>Num times clicked ${count}</div>
      <button @click=${this.clickCallback}>click me</button>`;
  }

  private clickCallback() {
    // No error because `this` refers to component
    this.count++;
  }
}
```

Lit 如何解決這些問題？

- `constructor` 不接受任何引數
- 所有 `@event` 繫結都會自動繫結至 `this`
- `this` 在絕大多數情況下，是指自訂元素的參照
- 類別屬性現在可以例項化為類別成員。這會清除以建構函式為基礎的實作項目

Reactive Controllers

Lit 中的**反應式控制器**是 Hooks 背後的主要概念。反應式控制器模式可共用有狀態的邏輯、將元件分割成更小的模組化位元，以及掛鉤至元素的更新生命週期。

反應式控制器是物件介面，可掛鉤至控制器主機 (例如 LitElement) 的更新生命週期。

如要進一步瞭解 Lit 元件生命週期，請參閱「**狀態和生命週期**」。

`ReactiveController` 和 `reactiveControllerHost` 的生命週期如下：

```
interface ReactiveController {
  hostConnected(): void;
  hostUpdate(): void;
  hostUpdated(): void;
  hostDisconnected(): void;
}
interface ReactiveControllerHost {
  addController(controller: ReactiveController): void;
  removeController(controller: ReactiveController): void;
  requestUpdate(): void;
  readonly updateComplete: Promise<boolean>;
}
```

建構反應式控制器並使用 `addController` 附加至主機後，系統會一併呼叫控制器和主機的生命週期。舉例來說，請回想「**狀態和生命週期**」一節中的時鐘範例：

```

import React from 'react';
import ReactDOM from 'react-dom';

class Clock extends React.Component {
  constructor(props) {
    super(props);
    this.state = {date: new Date()};
  }

  componentDidMount() {
    this.timerID = setInterval(
      () => this.tick(),
      1000
    );
  }

  componentWillUnmount() {
    clearInterval(this.timerID);
  }

  tick() {
    this.setState({
      date: new Date()
    });
  }

  render() {
    return (
      <div>
        <h1>Hello, world!</h1>
        <h2>It is {this.state.date.toLocaleTimeString()}.</h2>
      </div>
    );
  }
}

ReactDOM.render(
  <Clock />,
  document.getElementById('root')
);

```

在上述範例中，簡單時鐘會執行下列操作：

- 並顯示「Hello World! 「現在時間是」 並顯示時間
- 每秒更新時鐘
- 卸載時，系統會清除呼叫滴答的間隔

建構元件架構

首先，從元件類別宣告開始，然後新增 `render` 函式。

```

// Lit (TS) - index.ts
import {LitElement, html} from 'lit';
import {customElement} from 'lit/decorators.js';

@customElement('my-element')
class MyElement extends LitElement {
  render() {
    return html`
      <div>
        <h1>Hello, world!</h1>
        <h2>It is ${'time to get Lit'}.</h2>
      </div>
    `;
  }
}

// Lit (JS) - index.js
import {LitElement, html} from 'lit';

class MyElement extends LitElement {
  render() {
    return html`
      <div>
        <h1>Hello, world!</h1>
        <h2>It is ${'time to get Lit'}.</h2>
      </div>
    `;
  }
}

customElements.define('my-element', MyElement);

```

建構控制器

現在請切換至 `clock.ts`，為 `ClockController` 建立類別並設定 `constructor`：

```
// Lit (TS) - clock.ts
import {ReactiveController, ReactiveControllerHost} from 'lit';

export class ClockController implements ReactiveController {
  private readonly host: ReactiveControllerHost;

  constructor(host: ReactiveControllerHost) {
    this.host = host;
    host.addController(this);
  }

  hostConnected() {
  }

  private tick() {
  }

  hostDisconnected() {
  }
}

// Lit (JS) - clock.js
export class ClockController {
  constructor(host) {
    this.host = host;
    host.addController(this);
  }

  hostConnected() {
  }

  tick() {
  }

  hostDisconnected() {
  }
}
```

只要共用 `ReactiveController` 介面，就能以任何方式建構反應式控制器，但 Lit 團隊偏好使用具有 `constructor` 的類別，因為這類別可以接收 `ReactiveControllerHost` 介面，以及初始化控制器所需的任何其他屬性，適用於大多數基本情況。

現在您需要將 React 生命週期回呼轉換為控制器回呼。簡單來說，

- `componentDidMount`
 - 至 `LitElement` 的 `connectedCallback`
 - 控制器 `hostConnected`
- `ComponentWillUnmount`
 - 至 `LitElement` 的 `disconnectedCallback`
 - 控制器 `hostDisconnected`

如要進一步瞭解如何將 React 生命週期轉換為 Lit 生命週期，請參閱「狀態和生命週期」一節。

接著，實作 `hostConnected` 回呼和 `tick` 方法，並在 `hostDisconnected` 中清除間隔，如「狀態和生命週期」一節的範例所示。

```
// Lit (TS) - clock.ts
export class ClockController implements ReactiveController {
  private readonly host: ReactiveControllerHost;
  private interval = 0 as unknown as ReturnType<typeof setTimeout>;
  date = new Date();

  constructor(host: ReactiveControllerHost) {
    this.host = host;
    host.addController(this);
  }

  hostConnected() {
    this.interval = setInterval(() => this.tick(), 1000);
  }

  private tick() {
    this.date = new Date();
  }

  hostDisconnected() {
    clearInterval(this.interval);
  }
}

// Lit (JS) - clock.js
export class ClockController {
  interval = 0;
  host;
  date = new Date();

  constructor(host) {
    this.host = host;
    host.addController(this);
  }

  hostConnected() {
    this.interval = setInterval(() => this.tick(), 1000);
  }

  tick() {
    this.date = new Date();
  }

  hostDisconnected() {
    clearInterval(this.interval);
  }
}
```

使用控制器

如要使用時鐘控制器，請匯入控制器，並在 `index.ts` 或 `index.js` 中更新元件。

```
// Lit (TS) - index.ts
import {LitElement, html, ReactiveController, ReactiveControllerHost} from 'lit';
import {customElement} from 'lit/decorators.js';
import {ClockController} from './clock.js';

@customElement('my-element')
class MyElement extends LitElement {
  private readonly clock = new ClockController(this); // Instantiate

  render() {
    // Use controller
    return html`
      <div>
        <h1>Hello, world!</h1>
        <h2>It is ${this.clock.date.toLocaleTimeString()}.</h2>
      </div>
    `;
  }
}

// Lit (JS) - index.js
import {LitElement, html} from 'lit';
import {ClockController} from './clock.js';

class MyElement extends LitElement {
  clock = new ClockController(this); // Instantiate

  render() {
    // Use controller
    return html`
      <div>
        <h1>Hello, world!</h1>
        <h2>It is ${this.clock.date.toLocaleTimeString()}.</h2>
      </div>
    `;
  }
}

customElements.define('my-element', MyElement);
```

如要使用控制器，請傳遞控制器主機 (即 `<my-element>` 元件) 的參照來例項化控制器，然後在 `render` 方法中使用控制器。

在控制器中觸發重新算繪

請注意，畫面會顯示時間，但時間不會更新。這是因為控制器每秒都會設定日期，但主機不會更新。這是因為 `date` 現在會變更 `ClockController` 類別，而不是元件。也就是說，在控制器上設定 `date` 後，主機需要收到指令，透過 `host.requestUpdate()` 執行更新生命週期。

```
// Lit (TS & JS) - clock.ts / clock.js
private tick() {
  this.date = new Date();
  this.host.requestUpdate();
}
```

現在時鐘應該會開始計時！

如要進一步比較常見的 Hook 用途，請參閱「[進階主題 - Hook](#)」一節。

8. 兒童

在本節中，您將瞭解如何使用 slot 管理 Lit 中的子項。

Slot 和 Children

您可以透過插槽巢狀內嵌元件，藉此進行組合。

在 React 中，子項是透過 props 繼承。預設插槽為 `props.children`，而 `render` 函式會定義預設插槽的位置。例如：

```
const MyArticle = (props) => {  
  return <article>{props.children}</article>;  
};
```

請注意，`props.children` 是 React 元件，不是 HTML 元素。

在 Lit 中，子項會在 `render` 函式中以[插槽元素](#)組成。請注意，子項的繼承方式與 React 不同。在 Lit 中，子項是附加至插槽的 `HTMLElement`。這項附件稱為「預測」。

```
@customElement("my-article")  
export class MyArticle extends LitElement {  
  render() {  
    return html`  
      <article>  
        <slot></slot>  
      </article>  
    `;  
  }  
}
```

注意：在程式碼檢查點中，請注意 `index.html` 中的已插入元素

多個運算單元

在 React 中，新增多個插槽與繼承更多屬性本質上相同。

```
const MyArticle = (props) => {
  return (
    <article>
      <header>
        {props.headerChildren}
      </header>
      <section>
        {props.sectionChildren}
      </section>
    </article>
  );
};
```

同樣地，加入更多 `<slot>` 元素會在 Lit 中建立更多插槽。使用 `name` 屬性定義了多個位置：`<slot name="slot-name">`。孩子可以藉此聲明要分配到哪個時段。

```
@customElement("my-article")
export class MyArticle extends LitElement {
  render() {
    return html`
      <article>
        <header>
          <slot name="headerChildren"></slot>
        </header>
        <section>
          <slot name="sectionChildren"></slot>
        </section>
      </article>
    `;
  }
}
```

預設位置內容

如果沒有節點投影到該插槽，插槽就會顯示其子樹狀結構。節點投影至插槽時，該插槽不會顯示子樹狀結構，而是顯示投影的節點。

```
@customElement("my-element")
export class MyElement extends LitElement {
  render() {
    return html`
      <section>
        <div>
          <slot name="slotWithDefault">
            <p>
              This message will not be rendered when children are attached to this slot!
            <p>
          </slot>
        </div>
      </section>
    `;
  }
}
```

將孩子指派到空位

在 React 中，子項會透過元件的屬性指派給插槽。在以下範例中，React 元素會傳遞至 `headerChildren` 和 `sectionChildren` 屬性。

```
const MyNewsArticle = () => {
  return (
    <MyArticle
      headerChildren=<h3>Extry, Extry! Read all about it!</h3>
      sectionChildren=<p>Children are props in React!</p>
    />
  );
};
```

在 Lit 中，子項會使用 `slot` 屬性指派給插槽。

```
@customElement("my-news-article")
export class MyNewsArticle extends LitElement {
  render() {
    return html`
      <my-article>
        <h3 slot="headerChildren">
          Extry, Extry! Read all about it!
        </h3>
        <p slot="sectionChildren">
          Children are composed with slots in Lit!
        </p>
      </my-article>
    `;
  }
}
```

如果沒有預設的 slot (例如 `<slot>`)，也沒有具有 `name` 屬性的 slot (例如 `<slot name="foo">`) 與自訂元素子項的 `slot` 屬性 (例如 `<div slot="foo">`) 相符，則該節點不會投影，也不會顯示。

9. Refs

開發人員有時可能需要存取 `HTMLElement` 的 API。

在本節中，您將瞭解如何在 Lit 中取得元素參照。

React 參考資料

React 元件會轉譯為一系列函式呼叫，在叫用時建立虛擬 DOM。ReactDOM 會解讀這個虛擬 DOM，並轉譯 `HTMLElements`。

在 React 中，Refs 是記憶體中的空間，用於存放產生的 `HTMLElement`。

```
const RefsExample = (props) => {
  const inputRef = React.useRef(null);
  const onClick = React.useCallback(() => {
    inputRef.current?.focus();
  }, [inputRef]);

  return (
    <div>
      <input type="text" ref={inputRef} />
      <br />
      <button onClick={onClick}>
        Click to focus on the input above!
      </button>
    </div>
  );
};
```

在上述範例中，React 元件會執行下列操作：

- 算繪空白文字輸入欄位和帶有文字的按鈕
- 在點選按鈕時將焦點放在輸入內容上

初始算繪完成後，React 會透過 `ref` 屬性，將 `inputRef.current` 設為產生的 `HTMLInputElement`。

以 `@query` 點亮「參考資料」

Lit 接近瀏覽器，並在原生瀏覽器功能上建立非常薄的抽象層。

在 Lit 中，`refs` 的 React 對等項目是 `@query` 和 `@queryAll` 裝飾器傳回的 `HTMLElement`。

```

@customElement("my-element")
export class MyElement extends LitElement {
  @query('input') // Define the query
  inputEl!: HTMLInputElement; // Declare the prop

  // Declare the click event listener
  onClick() {
    // Use the query to focus
    this.inputEl.focus();
  }

  render() {
    return html`
      <input type="text">
      <br />
      <!-- Bind the click listener -->
      <button @click=${this.onClick}>
        Click to focus on the input above!
      </button>
    `;
  }
}

```

在上述範例中，Lit 元件會執行下列操作：

- 使用 `@query` 裝飾器在 `MyElement` 上定義屬性 (為 `HTMLInputElement` 建立 getter)。
- 宣告並附加名為 `onClick` 的點擊事件回呼。
- 在點選按鈕時將焦點移至輸入內容

在 JavaScript 中，`@query` 和 `@queryAll` 裝飾器會分別執行 `querySelector` 和 `querySelectorAll`。這是 JavaScript 版本的 `@query('input') inputEl!: HTMLInputElement;`

```

get inputEl() {
  return this.renderRoot.querySelector('input');
}

```

在 Lit 元件將 `render` 方法的範本提交至 `my-element` 的根目錄後，`@query` 裝飾器現在會允許 `inputEl` 傳回在算繪根目錄中找到的第一個 `input` 元素。如果 `@query` 找不到指定元素，就會傳回 `null`。

如果算繪根目錄中有多个 `input` 元素，`@queryAll` 會傳回節點清單。

10. 中介狀態

在本節中，您將瞭解如何在 Lit 中調解元件之間的狀態。

可重複使用的元件

React 會模擬功能性算繪管道，並採用由上而下的資料流程。家長透過道具為孩子提供狀態。孩子會透過屬性中的回呼與家長通訊。

```
const CounterButton = (props) => {  
  const label = props.step < 0  
    ? `- ${-1 * props.step}`  
    : `+ ${props.step}`;  
  
  return (  
    <button  
      onClick={() =>  
        props.addToCounter(props.step)}>{label}</button>  
  );  
};
```

在上述範例中，React 元件會執行下列動作：

- 根據 `props.step` 值建立標籤。
- 以 `+step` 或 `-step` 做為標籤，算繪按鈕
- 在點擊時呼叫 `props.addToCounter`，並以 `props.step` 做為引數，更新父項元件

雖然可以在 Lit 中傳遞回呼，但傳統模式有所不同。上例中的 React 元件可以改寫為下例中的 Lit 元件：

```

@customElement('counter-button')
export class CounterButton extends LitElement {
  @property({type: Number}) step: number = 0;

  onClick() {
    const event = new CustomEvent('update-counter', {
      bubbles: true,
      detail: {
        step: this.step,
      }
    });

    this.dispatchEvent(event);
  }

  render() {
    const label = this.step < 0
      ? `-${this.step}` // "- 1"
      : `+${this.step}`; // "+ 1"

    return html`
      <button @click=${this.onClick}>${label}</button>
    `;
  }
}

```

在上述範例中，Lit 元件會執行下列操作：

- 建立反應式屬性 `step`
- 在點擊時，傳送名為 `update-counter` 的自訂事件，並攜帶元素的 `step` 值

瀏覽器事件會從子項元素向上傳播至父項元素。孩子可以透過事件播送互動事件和狀態變化。React 本質上會以相反方向傳遞狀態，因此您很少會看到 React 元件以與 Lit 元件相同的方式，傳送及監聽事件。

與 React 在 props 中傳遞回呼的方式類似，Lit 範本可將回呼繫結至任何節點。不過，這無法為非 Lit 使用者提供良好的開發人員體驗（請參閱「[進階主題：JSX 和範本](#)」）。

有狀態元件

在 React 中，通常會使用 Hook 管理狀態。您可以重複使用 `CounterButton` 元件來建立 `MyCounter` 元件。請注意，`addToCounter` 會傳遞至 `CounterButton` 的兩個執行個體。


```

const MyCounter = (props) => {
  const [counterSum, setCounterSum] = React.useState(0);
  const addToCounter = useCallback(
    (step) => {
      setCounterSum(counterSum + step);
    },
    [counterSum, setCounterSum]
  );

  return (
    <div>
      <h3>&Sigma;; {counterSum}</h3>
      <CounterButton
        step={-1}
        addToCounter={addToCounter} />
      <CounterButton
        step={1}
        addToCounter={addToCounter} />
    </div>
  );
};

```

上述範例會執行下列操作：

- 建立 `count` 狀態。
- 建立回呼，將數字新增至 `count` 狀態。
- `CounterButton` 會在每次點擊時使用 `addToCounter` 更新 `count`，並在 `step` 上顯示。

您可以在 Lit 中實作類似的 `MyCounter`。請注意，`addToCounter` 不會傳遞至 `counter-button`。而是將回呼繫結為父項元素上 `@update-counter` 事件的事件監聽器。

```

@customElement("my-counter")
export class MyCounter extends LitElement {
  @property({type: Number}) count = 0;

  addToCounter(e: CustomEvent<{step: number}>) {
    // Get step from detail of event or via @query
    this.count += e.detail.step;
  }

  render() {
    return html`
      <div @update-counter="${this.addToCounter}">
        <h3>&Sigma; ${this.count}</h3>
        <counter-button step="-1"></counter-button>
        <counter-button step="1"></counter-button>
      </div>
    `;
  }
}

```

上述範例會執行下列操作：

- 建立名為 `count` 的反應式屬性，當值變更時，該屬性會更新元件
- 將 `addToCounter` 回呼繫結至 `@update-counter` 事件監聽器
- 透過新增 `update-counter` 事件的 `detail.step` 中找到的值，更新 `count`
- 透過 `step` 屬性設定 `counter-button` 的 `step` 值

在 Lit 中，使用反應式屬性從父項向子項廣播變更，是更常見的做法。同樣地，建議您使用瀏覽器的事件系統，從底部向上傳遞詳細資料。

這個方法遵循最佳做法，並符合 Lit 的目標，也就是為網頁元件提供跨平台支援。

11. 樣式

本節將說明 Lit 中的樣式設定。

樣式

Lit 提供多種元素樣式設定方式，以及內建解決方案。

內嵌樣式

Lit 支援內嵌樣式，以及繫結至這些樣式。

```
import {LitElement, html, css} from 'lit';
import {customElement} from 'lit/decorators.js';

@customElement('my-element')
class MyElement extends LitElement {
  render() {
    return html`
      <div>
        <h1 style="color:orange;">This text is orange</h1>
        <h1 style="color:rebeccapurple;">This text is rebeccapurple</h1>
      </div>
    `;
  }
}
```

在上述範例中，有 2 個標題，每個標題都有內嵌樣式。

現在請從 `border-color.js` 匯入邊框並繫結至橘色文字：

```
...
import borderColor from './border-color.js';

...

html`
  ...
  <h1 style="color:orange;${borderColor}">This text is orange</h1>
  ...`
```

每次都必須計算樣式字串可能有點麻煩，因此 Lit 提供指令來協助解決這個問題。

styleMap

`styleMap` 指令可讓您更輕鬆地使用 JavaScript 設定內嵌樣式。例如：

```

import {LitElement, html, css} from 'lit';
import {customElement, property} from 'lit/decorators.js';
import {styleMap} from 'lit/directives/style-map.js';

@customElement('my-element')
class MyElement extends LitElement {
  @property({type: String})
  color = '#000'

  render() {
    // Define the styleMap
    const headerStyle = styleMap({
      'border-color': this.color,
    });

    return html`
      <div>
        <h1
          style="border-style:solid;
          <!-- Use the styleMap -->
          border-width:2px;${headerStyle}">
          This div has a border color of ${this.color}
        </h1>
        <input
          type="color"
          @input=${e => (this.color = e.target.value)}
          value="#000">
        </div>
      `;
  }
}

```

上述範例會執行下列操作：

- 顯示帶有邊框的 `h1` 和顏色挑選器
- 將 `border-color` 變更為顏色挑選器中的值

此外，還有 `styleMap`，用於設定 `h1` 的樣式。`styleMap` 遵循的語法與 React 的 `style` 屬性繫結語法類似。

使用 `styleMap` 時，其他 JavaScript 繫結 (例如 `${...}`) 無法套用至相同的 `style` 屬性。

CSSResult

建議使用 `css` 標記的範本字面值設定元件樣式。

```
import {LitElement, html, css} from 'lit';
import {customElement} from 'lit/decorators.js';

const ORANGE = css`orange`;

@customElement('my-element')
class MyElement extends LitElement {
  static styles = [
    css`
      #orange {
        color: ${ORANGE};
      }

      #purple {
        color: rebeccapurple;
      }
    `,
  ];

  render() {
    return html`
      <div>
        <h1 id="orange">This text is orange</h1>
        <h1 id="purple">This text is rebeccapurple</h1>
      </div>
    `;
  }
}
```

上述範例會執行下列操作：

- 使用繫結宣告 CSS 標記範本常值
- 設定兩個具有 ID 的 `h1` 顏色

你會發現範本和 CSS 使用 `ids`。在 Lit 中，這項操作很安全，因為 DOM 和樣式預設會以 Shadow DOM 為範圍。

使用 `css` 範本代碼的好處：

- 每個類別與每個執行個體各剖析一次
- 實作時考量模組重複使用性
- 可輕鬆將樣式分成各自的檔案
- 與 CSS 自訂屬性 Polyfill 相容

此外，請注意 `index.html` 中的 `<style>` 標記：

```
<!-- index.html -->
<style>
  h1 {
    color: red !important;
  }
</style>
```

Lit 會將元件的樣式範圍限定在根目錄。也就是說，樣式不會洩漏到內外。如要將樣式傳遞至元件，Lit 團隊建議使用 [CSS 自訂屬性](#)，因為這些屬性可以穿透 Lit 樣式範圍。

樣式標記

您也可以直接在範本中內嵌 `<style>` 標記。瀏覽器會將這些樣式標記重複資料刪除，但如果將這些標記放在範本中，系統會剖析每個元件執行個體，而不是像 `css` 標記範本一樣剖析每個類別。此外，瀏覽器重複資料刪除 `CSSResult` 的速度也快得多。

連結標記

您也可以在範本中使用 `<link rel="stylesheet">` 設定樣式，但同樣不建議這麼做，因為這可能會導致未設定樣式的內容在初始階段閃爍 (FOUC)。

12. 進階主題 (選用)

本節將深入探討進階主題，並詳細說明 Lit 和 React 的實作方式。這個部分為選填

JSX 和範本

Lit 和虛擬 DOM

Lit-html 不包含傳統的虛擬 DOM，不會對每個節點進行差異比較。而是利用 ES2015 [標記範本常值](#) 規格內建的效能功能。標記範本常值是附加標記函式的範本常值字串。

範本字面值範例如下：

```
const str = 'string';
console.log(`This is a template literal ${str}`);
```

以下是標記範本字面值的範例：

```
const tag = (strings, ...values) => ({strings, values});
const f = (x) => tag`hello ${x} how are you`;
console.log(f('world')); // {strings: ["hello ", " how are you"], values: ["world"]}
console.log(f('world').strings === f(1 + 2).strings); // true
```

在上述範例中，標記是 `tag` 函式，而 `f` 函式會傳回標記範本常值的呼叫。

Lit 中許多效能魔法都來自於傳遞至標記函式的字串陣列具有相同指標 (如第二個 `console.log` 所示)。瀏覽器不會在每次叫用標記函式時重新建立新的 `strings` 陣列，因為它使用的是相同的範本常值 (即 AST 中的相同位置)。因此，Lit 的繫結、剖析和範本快取功能可以運用這些功能，而不會造成太多執行階段差異。

標記範本常值的內建瀏覽器行為，為 Lit 帶來相當大的效能優勢。傳統 Virtual DOM 大多數工作都是以 JavaScript 執行，不過，標記的範本常值會在瀏覽器的 C++ 中執行大部分的差異比較作業。

如要開始使用 React 或 Preact 的 HTML 標記範本字面值，Lit 團隊建議使用 [htm 程式庫](#)。

不過，如同 Google 程式碼研究室網站和幾個線上程式碼編輯器，您會發現標記的範本常值語法醒目顯示並不常見。部分 IDE 和文字編輯器預設支援這些標記，例如 [Atom](#) 和 [GitHub](#) 的程式碼區塊螢光筆。Lit 團隊也與社群密切合作，維護 [lit-plugin](#) 等專案，這個 VS Code 外掛程式會為 Lit 專案新增語法螢光標示、型別檢查和智慧感知功能。

Lit 和 JSX + React DOM

JSX 不會在瀏覽器中執行，而是使用前置處理器將 JSX 轉換為 JavaScript 函式呼叫 (通常是透過 Babel)。

舉例來說，Babel 會轉換以下程式碼：

```
const element = <div className="title">Hello World!</div>;
ReactDOM.render(element, mountNode);
```

改成：

```
const element = React.createElement('div', {className: 'title'}, 'Hello World!');
ReactDOM.render(element, mountNode);
```

React DOM 接著會採用 React 輸出內容，並轉換為實際 DOM (屬性、屬性、事件監聽器等)。

Lit-html 使用標記範本字面值，可在瀏覽器中執行，不必經過轉譯或預先處理。也就是說，如要開始使用 Lit，您只需要 HTML 檔案、ES 模組指令碼和伺服器。以下是完全可在瀏覽器中執行的指令碼：

```
<!DOCTYPE html>
<html>
  <head>
    <script type="module">
      import {html, render} from 'https://cdn.skypack.dev/lit';

      render(
        html`<div>Hello World!</div>`,
        document.querySelector('.root')
      )
    </script>
  </head>
  <body>
    <div class="root"></div>
  </body>
</html>
```

此外，由於 Lit 的範本系統 lit-html 並非使用傳統的虛擬 DOM，而是直接使用 DOM API，因此 Lit 2 縮小並壓縮後的大小不到 5 KB，相較於 React (2.8 KB) + react-dom (39.4 KB) 縮小並壓縮後的 40 KB，體積小了許多。

事件

React 使用合成事件系統。也就是說，react-dom 必須定義每個元件會使用的所有事件，並為每種節點提供對應的駝峰式大小寫事件監聽器。因此，JSX 沒有定義自訂事件監聽器的方法，開發人員必須使用 `ref`，然後強制套用監聽器。整合未考量 React 的程式庫時，這會導致開發人員體驗不佳，因此必須編寫 React 專用的包裝函式。

Lit-html 會直接存取 DOM 並使用原生事件，因此新增事件監聽器就像 `@event-name=${eventNameListener}` 一樣簡單。這表示新增事件監聽器和觸發事件時，執行的執行階段剖析作業較少。

元件和屬性

React 元件和自訂元素

在幕後，LitElement 會使用自訂元素封裝元件。就元件化而言，自訂元素會導致 React 元件之間出現一些取舍 (狀態和生命週期會在「狀態和生命週期」一節中進一步討論)。

自訂元素做為元件系統的優點包括：

- 瀏覽器原生支援，不需要任何工具
- 適用於所有瀏覽器 API，從 `innerHTML` 和 `document.createElement` 到 `querySelector`
- 通常適用於各種架構
- 可透過 `customElements.define` 延遲註冊，並「重整」DOM

與 React 元件相比，自訂元素有以下缺點：

- 無法建立自訂元素，而不自定義類別 (因此沒有類似 JSX 的函式元件)
- 必須包含結尾標記
 - 附註：雖然開發人員很方便，但瀏覽器供應商往往會後悔採用自閉標記規格，因此新版規格通常不會納入自閉標記
- 在 DOM 樹狀結構中導入額外節點，可能導致版面配置問題
- 必須透過 JavaScript 註冊

Lit 選擇使用自訂元素，而非專屬元素系統，是因為自訂元素內建於瀏覽器中，且 Lit 團隊認為跨框架優勢大於元件抽象層提供的優勢。事實上，Lit 團隊在 lit-ssr 領域的努力已克服 JavaScript 註冊的主要問題。此外，GitHub 等公司會利用自訂元素延遲註冊功能，逐步強化網頁並加入選用功能。

將資料傳遞至自訂元素

自訂元素常見的誤解是資料只能以字串形式傳遞。這種誤解可能源自於元素屬性只能寫成字串。雖然 Lit 會將字串屬性轉換為定義的型別，但自訂元素也可以接受複雜資料做為屬性。

舉例來說，如果指定下列 LitElement 定義：

```
// data-test.ts
import {LitElement, html} from 'lit';
import {customElement, property} from 'lit/decorators.js';

@customElement('data-test')
class DataTest extends LitElement {
  @property({type: Number})
  num = 0;

  @property({attribute: false})
  data = {a: 0, b: null, c: [html`<div>hello</div>`, html`<div>world</div>`]}

  render() {
    return html`
      <div>num + 1 = ${this.num + 1}</div>
      <div>data.a = ${this.data.a}</div>
      <div>data.b = ${this.data.b}</div>
      <div>data.c = ${this.data.c}</div>`;
  }
}
```

系統會定義原始反應式屬性 `num`，將屬性的字串值轉換為 `number`，然後導入 `attribute:false`，停用 Lit 的屬性處理作業。

以下說明如何將資料傳遞至這個自訂元素：

非 Lit 使用者可能會以這種方式使用這個元件，因為 Lit 範本內建屬性繫結。

```

<head>
  <script type="module">
    import './data-test.js'; // loads element definition
    import {html} from './data-test.js';

    const el = document.querySelector('data-test');
    el.data = {
      a: 5,
      b: null,
      c: [html`<div>foo</div>`,html`<div>bar</div>`]
    };
  </script>
</head>
<body>
  <data-test num="5"></data-test>
</body>

```

傳遞複雜屬性可能會降低非 Lit 使用者的開發人員體驗。通常複雜的屬性可以先解構，再傳遞至自訂元素。例如，`data-test` 元素可以修改為接受 `data.a` 和 `data.b` 做為個別屬性，而 `c` 可以算繪為 `<data-test>` 的直接子項。

狀態和生命週期

其他 React 生命週期回呼

`static getDerivedStateFromProps`

Lit 中沒有對應項目，因為 props 和 state 都是相同的類別屬性

`shouldComponentUpdate`

- Lit 對等項目為 `shouldUpdate`
- 與 React 不同，會在首次算繪時呼叫
- 功能與 React 的 `shouldComponentUpdate` 類似

`getSnapshotBeforeUpdate`

在 Lit 中，`getSnapshotBeforeUpdate` 類似於 `update` 和 `willUpdate`

`willUpdate`

- 在 `update` 前打過電話
- 與 `getSnapshotBeforeUpdate` 不同，`willUpdate` 會在 `render` 之前呼叫
- `willUpdate` 中反應式屬性的變更不會重新觸發更新週期
- 適合用來計算取決於其他屬性的屬性值，並用於更新程序的其餘部分
- 這個方法會在 SSR 的伺服器上呼叫，因此不建議在此存取 DOM

`update`

- 通話時間晚於 `willUpdate`

- 與 `getSnapshotBeforeUpdate` 不同，`update` 會在 `render` 之前呼叫
- 如果 `update` 中的反應式屬性在呼叫 `super.update` 之前變更，就不會重新觸發更新週期
- 在將算繪輸出內容提交至 DOM 之前，從元件周圍的 DOM 擷取資訊的絕佳位置
- 在 SSR 中，伺服器不會呼叫這個方法

其他 Lit 生命週期回呼

先前章節未提及幾個生命週期回呼，因為 React 中沒有類似的回呼。這 3 個子類型如下：

`attributeChangedCallback`

當元素其中一個 `observedAttributes` 變更時，系統會叫用這個函式。`observedAttributes` 和 `attributeChangedCallback` 都是自訂元素規格的一部分，Lit 會在幕後實作這些元素，為 Lit 元素提供屬性 API。

`adoptedCallback`

當元件移至新文件時 (例如從 `HTMLTemplateElement` 的 `documentFragment` 移至主要 `document`)，系統會叫用這個函式。這個回呼也是自訂元素規格的一部分，只有在元件變更文件時，才應用於進階用途。

其他生命週期方法和屬性

這些方法和屬性是您可以呼叫、覆寫或等待的類別成員，有助於操控生命週期程序。

`updateComplete`

這是 `Promise`，會在元素更新完成時解析，因為更新和算繪生命週期是非同步的。範例：

```
async nextButtonClicked() {
  this.step++;
  // Wait for the next "step" state to render
  await this.updateComplete;
  this.dispatchEvent(new Event('step-rendered'));
}
```

`getUpdateComplete`

這是應覆寫的方法，可自訂 `updateComplete` 的解析時間。當元件正在算繪子項元件，且算繪週期必須同步時，通常會發生這種情況。例如：

```
class MyElement extends LitElement {
  ...
  async getUpdateComplete() {
    await super.getUpdateComplete();
    await this.myChild.updateComplete;
  }
}
```

`performUpdate`

這個方法會呼叫更新生命週期回呼。一般來說，除非必須同步更新或自訂排程，否則不需要這麼做。

`hasUpdated`

如果元件至少更新過一次，這個屬性就會是 `true`。

`isConnected`

這個屬性是自訂元素規格的一部分，如果元素目前附加至主要文件樹狀結構，則為 `true`。

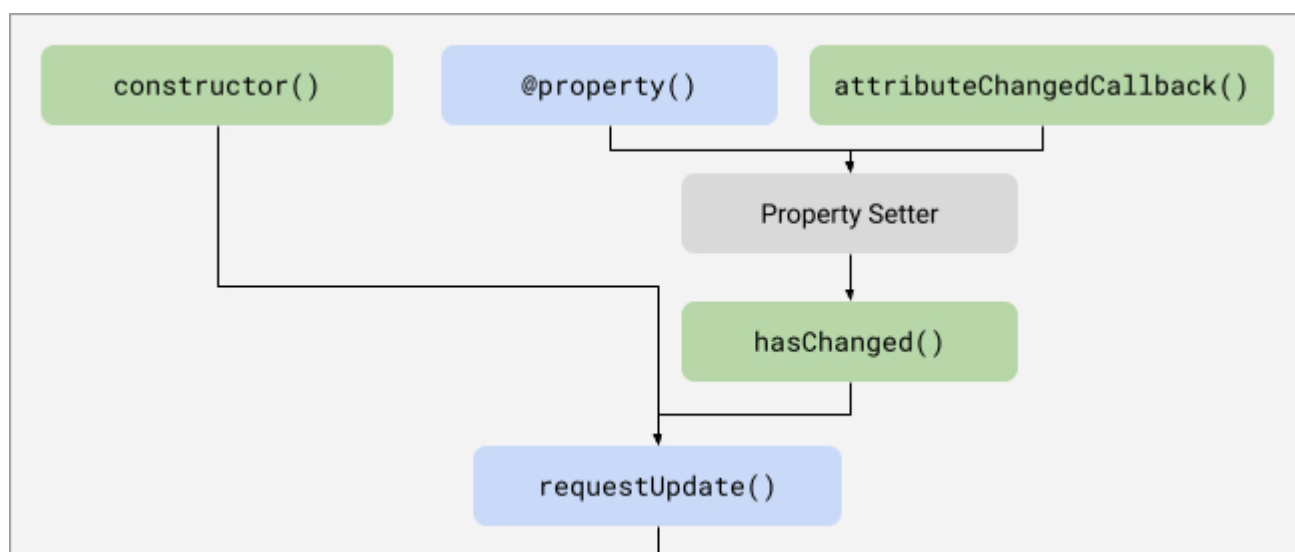
Lit 更新生命週期視覺化

更新生命週期分為 3 個部分：

- 更新前
- 更新
- 更新後

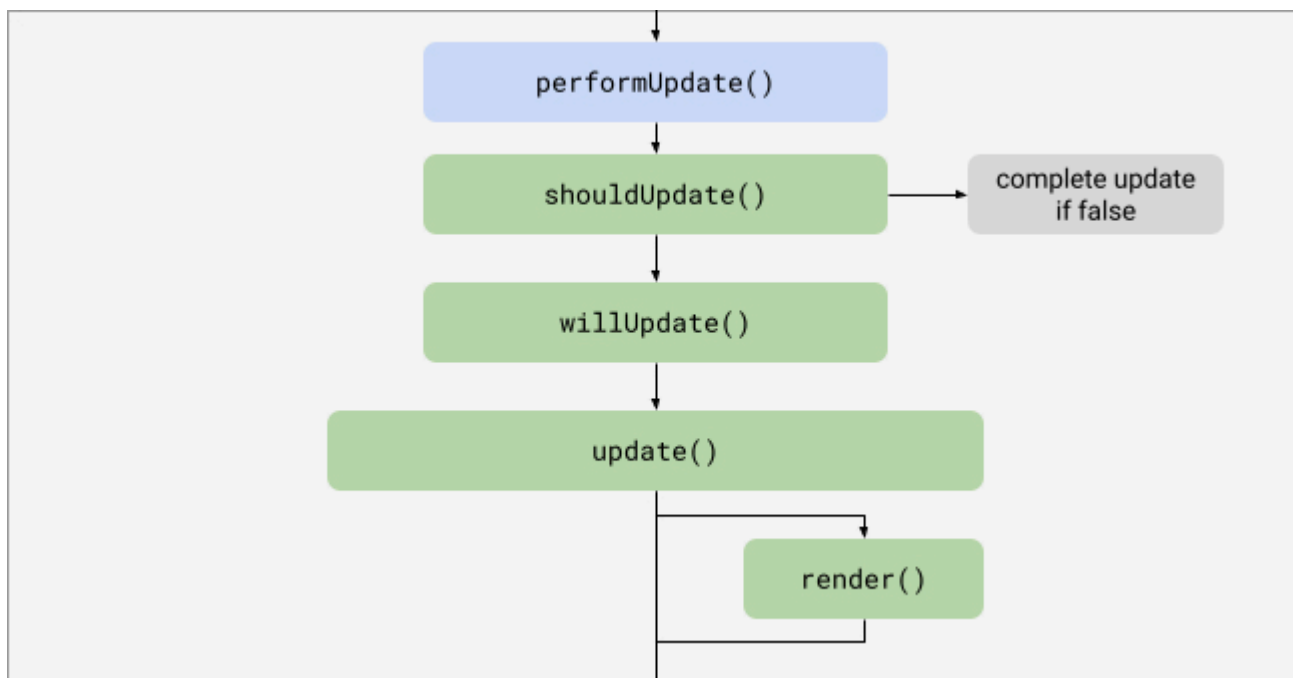
如要進一步瞭解尚未說明的生命週期回呼，請參閱「[進階主題 - 狀態和生命週期](#)」

更新前

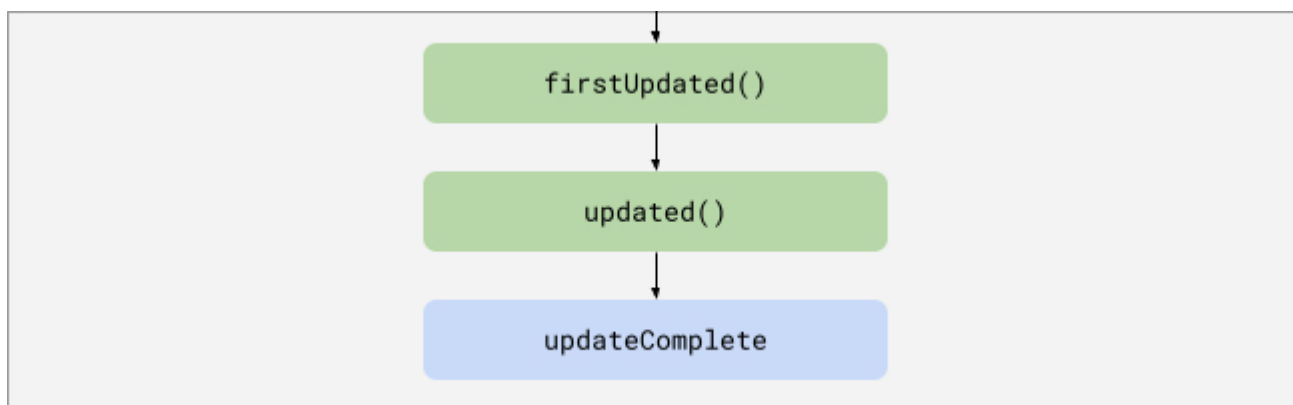


`requestUpdate` 後，系統會等待排定的更新。

更新



更新後



吊人胃口的情節片段

開場內容的重要性

React 導入 Hook，是為了簡化需要狀態的函式元件使用案例。在許多簡單情況下，使用 Hook 的函式元件往往比對應的類別元件更簡單易讀。不過，在導入非同步狀態更新，以及在掛鉤或效果之間傳遞資料時，掛鉤模式往往不夠用，而反應式控制器等以類別為基礎的解決方案則往往能發揮作用。

API 要求掛鉤和控制器

撰寫從 API 要求資料的 Hook 很常見。舉例來說，請參考[這個 React 函式元件](#)，該元件會執行下列操作：

- `index.tsx`
 - 顯示文字
 - 顯示 `useAPI` 的回覆
 - 使用者 ID + 使用者名稱
 - 錯誤訊息
 - 達到第 11 位使用者時會出現 404 錯誤 (這是設計所致)

- 如果 API 擷取作業中止，則會發生中止錯誤
- 載入訊息
- 顯示動作按鈕
 - 下一個使用者：擷取下一個使用者的 API
 - 取消：中止 API 擷取作業並顯示錯誤
- `useApi.tsx`
 - 定義 `useApi` 自訂 Hook
 - 會從 API 進行非同步擷取使用者物件
 - 發出：
 - 使用者名稱
 - 擷取作業是否正在載入
 - 所有錯誤訊息
 - 用於中止擷取作業的回呼
 - 如果卸載，則會中止進行中的擷取作業

以下是 [Lit + Reactive Controller](#) 的實作方式。

重點整理：

- 反應式控制器最類似於自訂 Hook
- 在回呼和效果之間傳遞無法顯示的資料
 - React 會使用 `useRef` 在 `useEffect` 和 `useCallback` 之間傳遞資料
 - Lit 使用私有類別屬性
 - React 基本上是在模仿私有類別屬性的行為

此外，如果您非常喜歡使用 Hook 的 React 函式元件語法，但又想使用 Lit 的相同免建構環境，Lit 團隊強烈建議使用 [Haunted](#) 程式庫。

兒童

預設位置

如果 HTML 元素未獲得 `slot` 屬性，系統會將其指派給預設的未命名插槽。在下列範例中，`MyApp` 會將一段文字插入具名插槽。另一個段落則會預設為未命名的「slot」。

```

@customElement("my-element")
export class MyElement extends LitElement {
  render() {
    return html`
      <section>
        <div>
          <slot></slot>
        </div>
        <div>
          <slot name="custom-slot"></slot>
        </div>
      </section>
    `;
  }
}

@customElement("my-app")
export class MyApp extends LitElement {
  render() {
    return html`
      <my-element>
        <p slot="custom-slot">
          This paragraph will be placed in the custom-slot!
        </p>
        <p>
          This paragraph will be placed in the unnamed default slot!
        </p>
      </my-element>
    `;
  }
}

```

預設插槽也會接受空白文字節點做為可指派的節點。這可能會導致預設範本發生問題。舉例來說，在 `<my-el>` `</my-el>` 中，`<slot><div>default</div><slot>` 不會顯示文字，因為有空白的文字節點。

運算單元更新

當插槽後代的結構變更時，系統會觸發 `slotchange` 事件。Lit 元件可以將事件監聽器繫結至 `slotchange` 事件。在下方範例中，`shadowRoot` 中找到的第一個 slot 會將 `assignedNodes` 記錄到 `slotchange` 的控制台中。

```
@customElement("my-element")
export class MyElement extends LitElement {
  onSlotChange(e: Event) {
    const slot = this.shadowRoot.querySelector('slot');
    console.log(slot.assignedNodes({flatten: true}));
  }

  render() {
    return html`
      <section>
        <div>
          <slot @slotchange="{this.onSlotChange}"></slot>
        </div>
      </section>
    `;
  }
}
```

`{flatten: true}` 設定會傳遞至 `assignedNodes` 和 `assignedElements`，因為 `<slot>` 可以投影至另一個 `<slot>`，而 `flatten` 會將節點投影樹狀結構扁平化。

Refs

生成參考資料

在呼叫 `render` 函式後，Lit 和 React 都會公開 `HTMLElement` 的參照。但建議您瞭解 React 和 Lit 如何組合 DOM，然後透過 Lit `@query` 裝飾器或 React 參照傳回。

React 是功能管道，可建立 React 元件，而非 `HTMLElements`。由於 Ref 是在 `HTMLElement` 算繪之前宣告，因此會分配記憶體空間。這就是為什麼您會看到 `null` 做為 Ref 的初始值，因為實際的 DOM 元素尚未建立 (或算繪)，也就是 `useRef(null)`。

ReactDOM 將 React 元件轉換為 `HTMLElement` 後，會在 `ReactComponent` 中尋找名為 `ref` 的屬性。如果可用，ReactDOM 會將 `HTMLElement` 的參照放置到 `ref.current`。

`LitElement` 會使用 lit-html 的 `html` 範本標記函式，在幕後組合 [Template Element](#)。在算繪後，`LitElement` 會將範本內容蓋印到自訂元素的 [shadow DOM](#)。shadow DOM 是由 shadow root 封裝的範圍 DOM 樹狀結構。接著，`@query` 裝飾器會為屬性建立 getter，基本上會在範圍內根層級上執行 `this.shadowRoot.querySelector`。

查詢多個元素

在以下範例中，`@queryAll` 裝飾器會將陰影根中的兩個段落做為 `NodeList` 傳回。


```
@customElement("my-element")
export class MyElement extends LitElement {
  @queryAll('p')
  paragraphs!: NodeList;

  render() {
    return html`
      <p>Hello, world!</p>
      <p>How are you?</p>
    `;
  }
}
```

基本上，`@queryAll` 會為 `paragraphs` 建立 getter，傳回 `this.shadowRoot.querySelectorAll()` 的結果。在 JavaScript 中，可以宣告 getter 來達到相同目的：

```
get paragraphs() {
  return this.renderRoot.querySelectorAll('p');
}
```

查詢變更元素

`@queryAsync` 修飾符更適合處理可根據其他元素屬性狀態變更的節點。

在下列範例中，`@queryAsync` 會找出第一個段落元素。不過，只有在 `renderParagraph` 隨機產生奇數時，系統才會算繪段落元素。`@queryAsync` 指令會傳回 Promise，在第一個段落可用時解析。

```
@customElement("my-dissappearing-paragraph")
export class MyDisappearingParagraph extends LitElement {
  @queryAsync('p')
  paragraph!: Promise<HTMLElement>;

  renderParagraph() {
    const randomNumber = Math.floor(Math.random() * 10)
    if (randomNumber % 2 === 0) {
      return "";
    }

    return html`<p>This checkbox is checked!`
  }

  render() {
    return html`
      ${this.renderParagraph()}
    `;
  }
}
```

中介狀態

在 React 中，慣例是使用回呼，因為狀態是由 React 本身調解。React 會盡量不依賴元素提供的狀態。DOM 只是轉譯過程的結果。

外部狀態

您可以搭配使用 Redux、MobX 或任何其他狀態管理程式庫和 Lit。

Lit 元件是在瀏覽器範圍中建立。因此，瀏覽器範圍內的所有程式庫都可供 Lit 使用。許多出色的程式庫都已建構完成，可運用 Lit 中現有的狀態管理系統。

以下是 Vaadin 的[系列文章](#)，說明如何在 Lit 元件中運用 Redux。

請參閱 Adobe 的 [lit-mobx](#)，瞭解大型網站如何在 Lit 中運用 MobX。

此外，請參閱 [Apollo Elements](#)，瞭解開發人員如何在網頁元件中加入 GraphQL。

Lit 可與原生瀏覽器功能搭配使用，且瀏覽器範圍內的大部分狀態管理解決方案都可用於 Lit 元件。

樣式

Shadow DOM

如要在自訂元素中以原生方式封裝樣式和 DOM，Lit 會使用 [Shadow DOM](#)。Shadow Root 會產生與主要文件樹狀結構分開的 shadow 樹狀結構。這表示大部分樣式都限定於這份文件。某些樣式 (例如顏色和其他字型相關樣式) 會洩漏。

陰影 DOM 也為 CSS 規格導入了新概念和選取器：

```
:host, :host(:hover), :host([hover]) {
  /* Styles the element in which the shadow root is attached to */
}

slot[name="title"]::slotted(*), slot::slotted(:hover), slot::slotted([hover]) {
  /*
   * Styles the elements projected into a slot element. NOTE: the spec only allows
   * styling the directly slotted elements. Children of those elements are not stylable.
   */
}
```

共用樣式

Lit 可透過 `css` 範本標記，輕鬆在元件之間以 `CSSTemplateResults` 形式共用樣式。例如：

```
// typography.ts
export const body1 = css`
  .body1 {
    ...
  }
`;

// my-el.ts
import {body1} from './typography.ts';

@customElement('my-el')
class MyEl Extends {
  static get styles = [
    body1,
    css`/* local styles come after so they will override bod1 */`
  ]

  render() {
    return html`<div class="body1">...</div>`
  }
}
```

主題設定

傳統主題設定通常是從上而下的樣式標記方法，因此陰影根會帶來一些挑戰。使用 Shadow DOM 的 Web Components 主題設定傳統做法，是透過 [CSS 自訂屬性](#)公開樣式 API。舉例來說，這是 Material Design 使用的模式：

```
.mdc-textfield-outline {
  border-color: var(--mdc-theme-primary, /* default value */ #...);
}
.mdc-textfield--input {
  caret-color: var(--mdc-theme-primary, #...);
}
```

接著，使用者可以套用自訂屬性值，變更網站的主題：

```
html {
  --mdc-theme-primary: #F00;
}
html[dark] {
  --mdc-theme-primary: #F88;
}
```

如果必須採用由上而下的主題設定，但無法公開樣式，您隨時可以覆寫 `createRenderRoot` 以傳回 `this`，藉此停用 Shadow DOM，然後將元件的範本算繪至自訂元素本身，而非附加至自訂元素的陰影根。這樣做會失去樣式封裝、DOM 封裝和插槽。

正式版

IE 11

如需支援舊版瀏覽器 (例如 IE 11)，您必須載入一些約 33 KB 的 Polyfill。詳情請參閱[這篇文章](#)。

條件式組合

Lit 團隊建議提供兩種不同的套件，分別適用於 IE 11 和新式瀏覽器。這麼做有幾個好處：

- 提供 ES 6 的速度較快，且適用於大多數用戶端
- 轉譯的 ES 5 會大幅增加軟體包大小
- 條件式套裝組合可兼顧兩者優點
 - 支援 IE 11
 - 新式瀏覽器不會變慢

如要進一步瞭解如何建構有條件放送的組合包，請參閱[說明文件網站](#)。
